Redis高级

删除策略

- 定时删除
  - 定时删除，CPU占用大
- 惰性删除
  - 过期数据二次访问后才删除，内存占用大
- 定期删除
  - 随机抽查删除，hz值决定了CPU和内存哪个占用大
- 淘汰策略
  - 内存不足时按策略删除
    - maxmemory ?mb 最大可用内存，默认为0不限制，一般设置物理机的50%以上
    - maxmemory-samples count 每次随机删除的数据个数
    - maxmemory-policy policy 删除策略
      - volatile 易失数据
        - volatile-lru 按最后一次使用时间
        - volatile-lfu 按最近使用次数
        - volatile-ttl 按即将过期的数据
        - volatile-random 随机
      - allkeys 全库数据
        - allkeys-lru 按最后一次使用时间
        - allkeys-lfu 按最近使用次数
        - allkeys-random 随机
      - no-enviction 故弃数据驱逐

主从复制

- 建立连接的流程
  - 设置master地址和端口并保存
  - 建立socket连接
  - 发送ping命令(定时器任务)
  - 身份验证
  - 发送slave端口信息

  三种方法
  - slaveof masterip masterport 客户端发送命令 —— slaveof no one 断开连接
  - redis-server -slaveof masterip masterport 启动服务器参数
  - slaveof masterip masterport 服务器配置
- 数据同步
  - 全量复制
    - 发送指令 psync2 <runid> <offset>
    - 执行bgsave
    - 第一个slave连接时创建命令缓冲区
    - 生成RDB文件通过socket发给slave
    - 接收RDB，清空数据，执行恢复过程
  - 部分复制
    - 发送命令告知RDB已经恢复完成
    - 发送复制缓冲区信息
    - 接收信息，执行bgrewriteaof，恢复数据
  - 心跳机制
    - master心跳
      - 内部指令 PING
      - 周期repl-ping-slave-period 默认10秒
      - INFO replication 获取slave最后一次连接时间间隔，lag项维持在0或1视为正常
    - slave心跳
      - 内部指令 REPLCONF ACK {offset}
      - 周期1秒
      - 汇报slave自己的复制偏移量，获取最新的数据变更指令
      - 判断master是否在线

      注意事项
      - slave多数掉线、延迟过高，master将会拒绝信息同步
      - min-slave-to-write 2
      - min-slave-max-lag 8
      - slave数量少于2个，或者所有slave延迟都大于等于8秒时，强制关闭master写功能，停止数据同步
      - slave数量和延迟由slave发送 REPLCONF ACK命令做确认
  - 注意事项
    - master数据同步应避开流量高峰期
    - 复制缓冲区太小会导致数据溢出，出现数据丢失并自动进行二次全量复制，死循环. repl-backlog-size ?mb 建议留下30%~50%的内存用于执行bgsave命令和创建复制缓冲区
    - 避免slave同步过程服务器响应阻塞或者数据不同步，建议关闭此期间的对外服务 slave-serve-stale-data yes | no
    - 数据同步阶段，master主动发送命令给slave，可以理解为是一个客户端
    - 多个slave同时对master请求同步，RDB只会有一份，但是带宽压力变大，可以适量错峰但是RDB会有多份
    - slave过多可以采用树状结构，但是数据一致性差

哨兵模式

- 建立过程配置
  - 检测master是否宕机，master宕机后选举slave为master
  - sentinel monitor master_name master_host master_port sentinel_number 设置监听的主服务器信息，sentinel_number表示参与投票的哨兵数量
  - sentinel down-after-milliseconds master_name million_seconds 设置宕机时长，控制是否进行主从切换
  - sentinel failover-timeout master_name million_seconds 设置故障切换操作的最大超时时长
  - sentinel parallel-syncs master_name sync_slave_number 设置主从切换后，同时进行数据同步的slave数量，越大网络资源要求越高，越小同步时间越长
- 监控阶段
  - 检查各个哨兵是否在线
  - 先向master发info查询其信息和名下slave
  - 再向slave发info查询slave具体信息
- 故障转移阶段
  - 单个哨兵监测到master无响应（主观下线）SRI_S_DOWN
  - 半数哨兵监测到master无响应（客观下线）SRI_O_DOWN
  - 哨兵选举
  - 选举master优先级
    - 不在线的
    - 响应慢的
    - 与原来master断开连接时间长的
    - 优先原则
      - 优先级
      - offset偏移量大
      - runid

Cluster集群

- 数据存储结构
  - 保存流程
    - CRC(key)%16384 计算结果为某台服务器中的槽位地址，读槽位不止存储一个key，而是大量key
  - 查询流程
    - 各个数据库相互通信，保存各个库中槽位的编号数据
    - 一次查询命中直接返回
    - 一次未命中，告知具体的位置
- 配置
  - cluster-enabled yes|no 是否启用cluster，加入cluster节点
  - cluster-config-file filename cluster配置文件名，该文件会自动生成，尽量手动命名否则会共用
  - cluster-node-timeout milliseconds 节点服务响应超时时间
  - cluster-migration-barrier min_slave_number master连接的slave最小数量

企业级解决方案

- 缓存预热
  - 现象：启动服务器后立刻宕机
  - 解决思路：系统启动前，提前将相关的缓存数据直接加载到缓存系统。避免在用户请求的时候，先查询数据库，然后再将数据缓存的问题！用户直接查询事先被预热的缓存数据！
- 缓存雪崩
  - 现象：短时间范围内，大量key集中过期导致一系列宕机
  - 解决思路
    - 更多的页面静态化处理
    - 构建多级缓存架构
      Nginx缓存+redis缓存+ehcache缓存
    - 检测Mysql严重耗时业务进行优化
      对数据库的瓶颈排查：例如超时查询、耗时较高事务等
    - 预警机制
    - 限流、降级
  - 具体解决方案
    - 切换为LFU淘汰策略
    - 调整过期的时间，使其错峰
    - 超热数据暂时使用永久key
    - 定期维护（自动+人工）
      对即将过期数据做访问量分析，确认是否延时，配合访问量统计，做热点数据的延时
    - 加锁：慎用！
- 缓存击穿
  - 现象：Redis中某个高热点key过期，该key访问量巨大
  - 解决方案
    - 增加热点key的时间
    - 设置热点key为永久
    - 高峰期来临前自动刷新有效期
    - 二级缓存，设置不同的过期时间错峰
    - 分布式锁
- 缓存穿透
  - 现象：黑客大量访问不存在的数据，跳过redis直接攻击数据库
  - 解决方案
    - 1、对查询结果为null的数据进行缓存（长期使用，定期清理），设定短时限，例如30-60秒，最高5分钟
    - 2、白名单策略
      提前把预热各种分类数据id对应的bitmaps，id作为bitmaps的offset，相当于设置了数据白名单。当加载正常数据时放行，加载异常数据时直接拦截（效率偏低）
      使用布隆过滤器（有关布隆过滤器的命中问题对当前状况可以忽略）
    - 3、实时监控redis命中率，提前预防，使用黑名单进行防控
    - 4、key加密
      问题出现后，临时启动防实业务key，对key进行业务层传输加密服务，设定校验程序，过来的key校验
    - 例如每天随机分配60个加密串，挑选2到3个，混淆到页面数据id中，发现访问key不满足规则，驳回数据访问